

MLE-STAR OpenAI Port

Detailed Work Plan

Porting MLE-STAR to a university OpenAI-compatible API

Minimum Viable Product (MVP) for limited budget

Date: June 14, 2026

1. Original MLE-STAR Architecture

Before describing the port, this section explains how the original MLE-STAR repository worked at commit 338cad9 ('Import MLE-STAR project'). The project was built on top of Google's Agent Development Kit (ADK) and used Gemini / Vertex AI as the underlying LLM.

1.1 — High-level flow

The original execution flow is a greedy, multi-stage pipeline. ADK loads a `root_agent`, which runs a `SequentialAgent` called `mle_pipeline_agent`. That pipeline runs four sub-agents in order:

1. Initialization: reads the task, searches the web for model candidates, generates code for each candidate, executes it, and keeps the best one.
2. Refinement: performs ablation studies to identify high-impact code blocks and refines them iteratively.
3. Ensemble: combines the best refined solutions into ensemble plans and evaluates them.
4. Submission: takes the best overall solution, adds test-data loading code, and writes `submission.csv`.

The diagram below shows the main file interactions:

```
machine_learning_engineering/agent.py
|
|-- root_agent (ADK Agent)
|   |
|   |-- mle_pipeline_agent (ADK SequentialAgent)
|       |
|       |-- initialization_agent ---> sub_agents/initialization/agent.py
|           |
|           |-- Uses shared_libraries/code_util.py to execute generated code
|           |-- Uses shared_libraries/debug_util.py for retry/debug loops
|           |-- Uses sub_agents/initialization/prompt.py for LLM instructions
|       |
|       |-- refinement_agent ---> sub_agents/refinement/agent.py
|           |
|           |-- Uses shared_libraries/code_util.py to execute ablation/refined code
|           |-- Uses shared_libraries/debug_util.py for retry/debug loops
|           |-- Uses sub_agents/refinement/prompt.py for LLM instructions
|       |
|       |-- ensemble_agent ---> sub_agents/ensemble/agent.py
|           |
|           |-- Uses shared_libraries/code_util.py to execute ensemble code
|           |-- Uses shared_libraries/debug_util.py for retry/debug loops
|           |-- Uses sub_agents/ensemble/prompt.py for LLM instructions
|       |
|       +-- submission_agent ---> sub_agents/submission/agent.py
|           |
|           |-- Uses shared_libraries/code_util.py to execute final solution
|           |-- Uses shared_libraries/debug_util.py for retry/debug loops
|           |-- Uses sub_agents/submission/prompt.py for LLM instructions
|
shared_libraries/config.py ---> Default configuration (paths, loops, timeouts, model name)
shared_libraries/common_util.py ---> Extracts text from LLM responses, copies files, sets seeds
shared_libraries/code_util.py ---> Writes code to disk, runs it via subprocess, parses scores
```

```
shared_libraries/debug_util.py ---> Builds retry/debug agent loops around code execution

tasks/<task_name>/ ---> Input data: train.csv, test.csv, task_description.txt
workspace/<task_name>/ ---> Generated workspace where scripts run and results are saved
```

1.2 — Role of the most important files

1.2.1 — machine_learning_engineering/agent.py

This is the entry point. It declares `root_agent` and `mle_pipeline_agent`. For ADK tooling, the top-level agent must be named `root_agent`. `mle_pipeline_agent` is a `SequentialAgent` that chains the four sub-agents and saves the final state to `final_state.json`.

1.2.2 — machine_learning_engineering/__init__.py

Sets Google Cloud / Vertex AI environment variables and imports the agent module so that ADK can discover `root_agent`. In the MVP this file will be replaced by OpenAI client initialization.

1.2.3 — shared_libraries/config.py

Contains `DefaultConfig`, a dataclass with all pipeline parameters: `data_dir`, `workspace_dir`, `task_name`, model name, timeout, number of solutions, loop rounds, and flags for optional checks. All sub-agents read from this configuration.

1.2.4 — shared_libraries/common_util.py

Provides `get_text_from_response` to extract text from ADK response objects, `set_random_seed` for reproducibility, and `copy_file` for file operations.

1.2.5 — shared_libraries/code_util.py

The execution engine. `run_python_code` writes generated code to a file and runs it with `subprocess`. `extract_performance_from_text` parses the 'Final Validation Performance: <score>' line. `evaluate_code` ties everything together: it decides the file path, checks safety conditions, runs the code, and stores the result (returncode, stdout, stderr, score) in state.

1.2.6 — shared_libraries/debug_util.py

Builds nested retry and debug loops. `get_run_and_debug_agent` creates ADK `LoopAgents` and `SequentialAgents` that generate code, execute it, summarize errors, and ask a debug agent to fix the code. `get_code_from_response` extracts the code block from the LLM output and triggers `evaluate_code`.

1.2.7 — sub_agents/initialization/agent.py

Implements the initialization phase: `prepare_task` copies config into state and reads `task_description.txt`; `create_workspace` sets up the workspace directory; model retrieval uses `google_search` to find candidate models; `model_eval` agents generate and execute code for each candidate; `rank_candidate_solutions` selects the best one.

1.2.8 — sub_agents/refinement/agent.py

Implements ablation-based refinement. It runs ablation studies, extracts a high-impact code block, generates improvement plans, implements them, and keeps only the variant with the best score.

1.2.9 — sub_agents/ensemble/agent.py

Builds ensemble plans that combine multiple refined solutions, executes the plans, and keeps the best-scoring ensemble.

1.2.10 — sub_agents/submission/agent.py

Takes the best solution (refined or ensemble), injects test-data loading logic, executes it, and produces the final submission.csv.

1.2.11 — sub_agents/*/prompt.py

Each sub-agent has its own prompt module. The prompts are carefully engineered to force the LLM to output a single, self-contained Python script, to load data from ./input, to avoid exit(), and to print 'Final Validation Performance: <score>'.

1.3 — State management

ADK passes a `callback_context.state` object through every callback and instruction provider. This state dictionary persists across the entire pipeline and holds: task description, workspace paths, candidate models, generated code, execution results, scores, counters, and configuration. Sub-agents read from state to build prompts and write results back so later stages can use them.

1.4 — Why the original does not run with the university API

The entire orchestration is expressed in ADK primitives (Agent, SequentialAgent, ParallelAgent, LoopAgent, `callback_context`). The LLM client is hidden inside ADK and configured for Vertex AI / Gemini. To use the university OpenAI-compatible API, the ADK runtime must be replaced by a custom runner while preserving the logic, prompts, code execution, and state management.

2. Executive Summary

This document describes the detailed implementation plan for porting the MLE-STAR project from Google ADK / Gemini to a university-provided OpenAI-compatible API. The goal is to produce a functional Minimum Viable Product (MVP) that can generate, execute, and score Python machine learning code for the California Housing task, while respecting a very limited budget (25 EUR).

Because the full MLE-STAR pipeline consumes a large number of tokens (multiple model candidates, ablation studies, ensemble plans, debug loops), the MVP drastically reduces the scope: one solution, one model candidate, no web search, no refinement, no ensemble, minimal debug loop.

The work is organized into phases: environment preparation, runner infrastructure, adaptation of shared libraries, adaptation of sub-agents, reconstruction of the main entry point, configuration tuning, testing, and cleanup.

3. Pre-Implementation Design Decisions

Before writing code, the following parameters are frozen for the MVP:

Parameter	Original Value	MVP Value	Rationale
num_solutions	2	1	Only one solution branch.
num_model_candidates	2	1	Only one model candidate.
inner_loop_round	1	0	No refinement.
outer_loop_round	1	0	No refinement.
ensemble_loop_round	1	0	No ensemble.
max_retry	10	1	Few generation retries.
max_debug_round	5	1	Few debug attempts.
max_rollback_round	2	0	No rollback.
use_data_leakage_checker	False	False	Keep disabled.
use_data_usage_checker	False	False	Keep disabled.

Resulting pipeline:

```
prepare task -> summarize task -> generate initial code -> execute -> (debug if needed) -> generate submission.csv
```

4. Phase 1 — Environment Preparation

3.1 — Secure Credential Handling

Keep .env.example as a template. Ensure .env is listed in .gitignore. Do not commit the real .env file. The application will load credentials via python-dotenv at runtime.

- Input: .env.example template.
- Output: .gitignore updated, .env left untouched in workspace.
- Files affected: .env.example, .gitignore.

3.2 — Clean Working Tree

Remove files that depend on ADK and will break imports in the MVP:

- tests/test_agents.py
- deployment/deploy.py
- eval/ directory
- prueba_api.py (if present)
- machine_learning_engineering/__pycache__/

Rationale: these files reference ADK root_agent or ADK evaluators and are not usable in the new runtime.

3.3 — Adjust pyproject.toml

Dependency changes:

- Remove google-adk, google-genai, google-cloud-aiplatform.
- Remove duplicate python-dotenv entry.
- Keep openai>=1.14.0, python-dotenv, pandas, scikit-learn, numpy, torch.
- Keep skrub (future roadmap, not used in MVP).
- Regenerate uv.lock with uv lock if uv is used.
- Fix tool.ruff.extend path if it points outside the project.

5. Phase 2 — OpenAI Runner Infrastructure

4.1 — Rewrite `machine_learning_engineering/__init__.py`

Purpose: create a global OpenAI client and expose `MODEL_NAME`.

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

API_KEY = os.environ.get("OPENAI_API_KEY")
API_BASE = os.environ.get("OPENAI_API_BASE")
MODEL_NAME = os.environ.get("ROOT_AGENT_MODEL", "meta-llama-3.1-8b-instruct")

if not API_KEY or not API_BASE:
    raise ValueError("OPENAI_API_KEY and OPENAI_API_BASE are required.")

client = OpenAI(api_key=API_KEY, base_url=API_BASE)
```

Inputs: environment variables `OPENAI_API_KEY`, `OPENAI_API_BASE`, `ROOT_AGENT_MODEL`.

Outputs: client (OpenAI instance), `MODEL_NAME` (str).

4.2 — Create `machine_learning_engineering/runner.py`

This module replaces the ADK runtime. It defines the state object and functions to call the LLM and orchestrate agents.

4.2.1 — class `AgentState(dict)`

Replaces `callback_context.state` from ADK.

```
class AgentState(dict):
    def get(self, key, default=None):
        return super().get(key, default)

    def to_dict(self):
        return dict(self)
```

Inputs: none (constructor). Outputs: dict-like state object.

4.2.2 — def `llm_call(messages, temperature=0.0, model=None)`

Calls the OpenAI-compatible chat completions endpoint.

```
def llm_call(messages, temperature=0.0, model=None):
    response = client.chat.completions.create(
        model=model or MODEL_NAME,
        messages=messages,
        temperature=temperature,
    )
    return response
```

Inputs: messages (list[dict]), temperature (float), model (str, optional).

Output: OpenAI chat completion response object.

4.2.3 — **def run_agent(state, agent_name, instruction, before_model=None, after_model=None)**

Executes a single LLM agent step.

```
def run_agent(state, agent_name, instruction, before_model=None, after_model=None, temperature=0.0):
    # Skip hook
    if before_model:
        skip_response = before_model(state, agent_name)
        if skip_response is not None:
            response = skip_response
        else:
            instruction_str = instruction(state, agent_name) if callable(instruction) else instruction
            messages = [{"role": "user", "content": instruction_str}]
            response = llm_call(messages, temperature=temperature)
    else:
        instruction_str = instruction(state, agent_name) if callable(instruction) else instruction
        messages = [{"role": "user", "content": instruction_str}]
        response = llm_call(messages, temperature=temperature)

    if after_model:
        after_model(state, agent_name, response)
```

Inputs:

- state: AgentState instance.
- agent_name: str, unique agent identifier.
- instruction: str or callable(state, agent_name) -> str.
- before_model: optional callable(state, agent_name) -> response | None.
- after_model: optional callable(state, agent_name, response) -> None.
- temperature: float.

Output: None (modifies state via callbacks).

4.2.4 — **def run_sequential(state, agents, description="")**

Runs a list of agent definitions sequentially.

```
def run_sequential(state, agents, description=""):
    for agent in agents:
        run_agent(state, **agent)
```

Inputs: state (AgentState), agents (list[dict]), description (str). Output: None.

4.2.5 — **def run_loop(state, agent, max_iterations=1, continue_fn=None)**

Runs an agent repeatedly up to max_iterations or until continue_fn returns False.

```
def run_loop(state, agent, max_iterations=1, continue_fn=None):
    for i in range(max_iterations):
        run_agent(state, **agent)
        if continue_fn and not continue_fn(state, i):
            break
```

Inputs: state, agent (dict), max_iterations (int), continue_fn (callable). Output: None.

4.2.6 — def run_parallel(state, agents, description="")

For the MVP, run_parallel will be implemented as sequential execution to save tokens and simplify debugging.

Inputs/Same as run_sequential. Output: None.

6. Phase 3 — Adapt Shared Libraries

5.1 — Adapt machine_learning_engineering/shared_libraries/common_util.py

Changes:

- Remove import google.adk.models.llm_response.
- Keep numpy if needed for seeding, but torch import can be removed or kept lazily.
- Rewrite get_text_from_response to accept an OpenAI response.

5.1.1 — def get_text_from_response(response) -> str

```
def get_text_from_response(response):
    return response.choices[0].message.content
```

Input: OpenAI chat completion response. Output: str.

5.2 — Adapt machine_learning_engineering/shared_libraries/code_util.py

Main change: remove callback_context dependency, pass (state, agent_name) instead.

5.2.1 — def get_updated_suffix(state, agent_name) -> str

Replaces get_updated_suffix(callback_context).

```
def get_updated_suffix(state, agent_name):
    if agent_name.startswith("model_eval"):
        model_id = agent_name.split("_")[-1]
        task_id = agent_name.split("_")[-2]
        suffix = f"{task_id}_{model_id}"
    elif agent_name.startswith("submission"):
        suffix = ""
    # ... (other branches identical to original)
    return suffix
```

Inputs: state (AgentState), agent_name (str). Output: suffix (str).

5.2.2 — def evaluate_code(state, agent_name) -> None

Replaces evaluate_code(callback_context).

```
def evaluate_code(state, agent_name):
    lower = state.get("lower", True)
    exec_timeout = state.get("exec_timeout", 1800)
    suffix = get_updated_suffix(state, agent_name)
    code_state_key = get_code_state_key(agent_name, suffix)
    raw_code = state.get(code_state_key, "")
    # Determine py_filepath based on agent_name (same logic as original)
    # ...
    if get_run_code_condition(agent_name, raw_code):
        workspace_dir = state.get("workspace_dir", "")
        task_name = state.get("task_name", "")
        run_cwd = os.path.join(workspace_dir, task_name, task_id)
```

```

result_dict = run_python_code(
    code_text=raw_code,
    run_cwd=run_cwd,
    py_filepath=py_filepath,
    exec_timeout=exec_timeout,
)
# Extract score or ablation result (same logic as original)
# ...
else:
    result_dict = {}
code_execution_result_state_key = get_code_execution_result_state_key(agent_name, suffix)
state[code_execution_result_state_key] = result_dict

```

Inputs: state (AgentState), agent_name (str). Output: None (modifies state).

5.2.3 — Functions kept unchanged

- run_python_code(code_text, run_cwd, py_filepath, exec_timeout)
- extract_performance_from_text(text)
- get_name_with_prefix_and_suffix(base_name, prefix, suffix)
- get_code_state_key(agent_name, suffix)
- get_code_execution_result_state_key(agent_name, suffix)
- get_run_code_condition(agent_name, raw_code)

5.3 — Simplify machine_learning_engineering/shared_libraries/debug_util.py

The original debug_util builds nested ADK LoopAgents. For the MVP we replace it with a single helper.

5.3.1 — def get_code_from_response(state, agent_name, response, do_eval=True) -> None

```

def get_code_from_response(state, agent_name, response, do_eval=True):
    response_text = common_util.get_text_from_response(response)
    code = response_text.replace("```python", "").replace("```", "")
    suffix = code_util.get_updated_suffix(state, agent_name)
    code_state_key = code_util.get_code_state_key(agent_name, suffix)

    if agent_name.startswith("check_data_use"):
        if "All the provided information is used" in code:
            state[f"check_data_use_finish_{suffix}"] = True
        if state.get(f"check_data_use_finish_{suffix}", False):
            return
        new_code = code
    elif agent_name.startswith("plan_implement"):
        if "debug" not in agent_name:
            task_id = agent_name.split("_")[-1]
            step = state.get(f"refine_step_{task_id}", 0)
            code_block = state.get(f"refine_code_block_{step}_{task_id}", "")
            prev_code = state.get(f"train_code_{step}_{task_id}", "")
            new_code = prev_code.replace(code_block, code)
        else:
            new_code = code
    else:
        new_code = code

    state[code_state_key] = new_code
    if do_eval:
        code_util.evaluate_code(state, agent_name)

```

Inputs: state, agent_name, response, do_eval. Output: None.

5.3.2 — def run_and_debug(state, agent_name, instruction, before_model=None, max_retry

New helper that replaces get_run_and_debug_agent.

```
def run_and_debug(state, agent_name, instruction, before_model=None, max_retry=1, max_debug=1):
    for attempt in range(max_retry):
        run_agent(
            state,
            agent_name,
            instruction,
            before_model=before_model,
            after_model=lambda s, n, r: get_code_from_response(s, n, r, do_eval=True),
            temperature=1.0,
        )
        result_key = code_util.get_code_execution_result_state_key(
            agent_name, code_util.get_updated_suffix(state, agent_name)
        )
        result = state.get(result_key, {})
        if result.get("returncode", 1) == 0 and result.get("score") is not None:
            return

    # Debug loop
    for debug_round in range(max_debug):
        bug_summary_key = code_util.get_name_with_prefix_and_suffix(
            "bug_summary", agent_name, code_util.get_updated_suffix(state, agent_name)
        )
        state[bug_summary_key] = result.get("stderr", "")
        debug_instruction = debug_prompt.BUG_REFINE_INSTR.format(
            task_description=state.get("task_description", ""),
            code=state.get(code_util.get_code_state_key(agent_name, code_util.get_updated_suffix(state, agent_name)), ""),
            bug=state.get(bug_summary_key, ""),
        )
        run_agent(
            state,
            f"{agent_name}_debug_{debug_round}",
            debug_instruction,
            after_model=lambda s, n, r: get_code_from_response(s, agent_name, r, do_eval=True),
            temperature=0.0,
        )
        result = state.get(result_key, {})
        if result.get("returncode", 1) == 0 and result.get("score") is not None:
            return
```

Inputs:

- state: AgentState.
- agent_name: str.
- instruction: str or callable.
- before_model: optional callable.
- max_retry: int.
- max_debug: int.

Output: None (modifies state).

7. Phase 4 — Adapt Sub-Agents

6.1 — Adapt machine_learning_engineering/sub_agents/initialization/agent.py

Replace ADK agent declarations with functions callable by the runner.

6.1.1 — def prepare_task(state) -> None

Copies config into state and reads task_description.txt.

```
def prepare_task(state):
    config_dict = dataclasses.asdict(config.CONFIG)
    for key in config_dict:
        state[key] = config_dict[key]
    state["start_time"] = time.time()
    common_util.set_random_seed(state["seed"])
    task_name = state.get("task_name", "")
    data_dir = state.get("data_dir", "")
    task_description = open(
        os.path.join(data_dir, task_name, "task_description.txt")
    ).read()
    state["task_description"] = task_description
```

Input: state. Output: None.

6.1.2 — def create_workspace(state, task_id) -> None

Creates workspace/<task_name>/<task_id>/input and copies task files.

Input: state, task_id. Output: None.

6.1.3 — def get_task_summary(state, response) -> None

Extracts summary from LLM response and stores task_summary.

Input: state, response. Output: None.

6.1.4 — def get_model_candidates(state, response, task_id) -> None

Extracts model list from response and stores init_{task_id}_model_1. Since there is no web search, the prompt will be adjusted to ask the LLM to propose a model directly.

Input: state, response, task_id. Output: None.

6.1.5 — def get_model_eval_agent_instruction(state, agent_name) -> str

Builds the MODEL_EVAL_INSTR prompt.

```
def get_model_eval_agent_instruction(state, agent_name):
    task_description = state.get("task_description", "")
    model_id = agent_name.split("_")[-1]
    task_id = agent_name.split("_")[-2]
    model_description = state.get(
        f"init_{task_id}_model_{model_id}", {}
    )
```

```

).get("model_description", "")
return prompt.MODEL_EVAL_INSTR.format(
    task_description=task_description,
    model_description=model_description,
)

```

Inputs: state, agent_name. Output: str.

6.1.6 — def rank_candidate_solutions(state, task_id) -> None

Sorts candidates by score, stores base_solution, best_score, and train_code_0.

Input: state, task_id. Output: None.

6.1.7 — def run_initialization_pipeline(state) -> None

Orchestrates the entire initialization phase.

```

def run_initialization_pipeline(state):
    task_id = 1
    prepare_task(state)
    create_workspace(state, task_id)

    # Task summarization
    run_agent(
        state,
        "task_summarization_agent",
        prompt.SUMMARIZATION_AGENT_INSTR,
        after_model=get_task_summary,
        temperature=0.0,
    )

    # Model retrieval (no web search, LLM proposes model)
    run_agent(
        state,
        f"model_retriever_agent_{task_id}",
        get_model_retriever_agent_instruction,
        after_model=lambda s, r: get_model_candidates(s, r, task_id),
        temperature=1.0,
    )

    # Generate and execute initial code
    run_and_debug(
        state,
        f"model_eval_{task_id}_1",
        get_model_eval_agent_instruction,
        max_retry=state.get("max_retry", 1),
        max_debug=state.get("max_debug_round", 1),
    )

    # Rank and select best
    rank_candidate_solutions(state, task_id)
    select_best_solution(state, task_id)

```

Input: state. Output: None.

6.2 — Adapt machine_learning_engineering/sub_agents/submission/agent.py

6.2.1 — def get_submission_and_debug_agent_instruction(state, agent_name) -> str

Builds ADD_TEST_FINAL_INSTR using the best refined solution.

```
def get_submission_and_debug_agent_instruction(state, agent_name):
    task_description = state.get("task_description", "")
    final_solution = state.get("train_code_0_1", "")
    return prompt.ADD_TEST_FINAL_INSTR.format(
        task_description=task_description,
        code=final_solution,
    )
```

Inputs: state, agent_name. Output: str.

6.2.2 — def run_submission_pipeline(state) -> None

Generates final_solution.py and submission.csv.

```
def run_submission_pipeline(state):
    run_and_debug(
        state,
        "submission",
        get_submission_and_debug_agent_instruction,
        max_retry=state.get("max_retry", 1),
        max_debug=state.get("max_debug_round", 1),
    )
```

Input: state. Output: None.

6.3 — Deactivate refinement and ensemble

For the MVP, do not adapt refinement/agent.py and ensemble/agent.py. They will remain in the repository but will not be imported or executed by the main pipeline.

8. Phase 5 — Reconstruct Main Entry Point

7.1 — New machine_learning_engineering/agent.py

Replaces the ADK root_agent and mle_pipeline_agent with a plain Python orchestrator.

```
from machine_learning_engineering.shared_libraries import config
from machine_learning_engineering.runner import AgentState
from machine_learning_engineering.sub_agents.initialization import agent as init_agent
from machine_learning_engineering.sub_agents.submission import agent as submission_agent

def run_pipeline(task_name=None, data_dir=None, workspace_dir=None):
    state = AgentState()
    cfg = config.CONFIG
    if task_name:
        state["task_name"] = task_name
    if data_dir:
        state["data_dir"] = data_dir
    if workspace_dir:
        state["workspace_dir"] = workspace_dir

    init_agent.run_initialization_pipeline(state)
    submission_agent.run_submission_pipeline(state)

    return state

if __name__ == "__main__":
    state = run_pipeline()
    print("Pipeline finished.")
    print("Best score:", state.get("best_score_1"))
    print("Submission result:", state.get("submission_code_exec_result", {}))
```

Inputs: optional task_name, data_dir, workspace_dir overrides.

Output: AgentState containing final results.

9. Phase 6 — Reduced Configuration

8.1 — Modify machine_learning_engineering/shared_libraries/config.py

Set MVP values in DefaultConfig:

```
num_solutions: int = 1
num_model_candidates: int = 1
inner_loop_round: int = 0
outer_loop_round: int = 0
ensemble_loop_round: int = 0
max_retry: int = 1
max_debug_round: int = 1
max_rollback_round: int = 0
```

```
exec_timeout: int = 300  
agent_model: str = os.environ.get("ROOT_AGENT_MODEL", "meta-llama-3.1-8b-instruct")
```

10. Phase 7 — Testing

9.1 — Create scripts/test_pipeline.py

A simple script that runs the pipeline and reports key metrics.

```
from machine_learning_engineering.agent import run_pipeline

state = run_pipeline()
print("Best score:", state.get("best_score_1"))
print("Submission exec result:", state.get("submission_code_exec_result", {}))
print("Final state saved to:", state.get("workspace_dir"))
```

9.2 — Execute with California Housing

Command:

```
python -m machine_learning_engineering.agent
```

Validation checklist:

1. No import errors.
2. Workspace directory created: workspace/california-housing-prices/.
3. init_code_1_1.py generated and executed.
4. Stdout contains 'Final Validation Performance: X.XXXX'.
5. final_solution.py and submission.csv generated.
6. final_state.json saved.

11. Phase 8 — Cleanup and Documentation

10.1 — Remove obsolete files

- tests/test_agents.py
- deployment/deploy.py
- eval/ directory
- machine_learning_engineering/sub_agents/refinement/ (or leave inactive)
- machine_learning_engineering/sub_agents/ensemble/ (or leave inactive)

10.2 — Update README.md

Add sections for:

- How to configure .env.

- How to run the MVP pipeline.
- Budget warning and how to adjust config for larger runs.

12. Summary of New and Modified Functions

File	Function/Class	Type	Inputs	Outputs
__init__.py	client, MODEL_NAME	Global vars	Env vars	—
runner.py	AgentState	Class	—	dict-like
runner.py	llm_call	Function	messages, temperature, model	OpenAI response
runner.py	run_agent	Function	state, agent_name	Instruction, ...
runner.py	run_sequential	Function	state, agents	None
runner.py	run_loop	Function	state, agent, max_iterations	None
common_util.py	get_text_from_response	Modified	response	str
code_util.py	get_updated_suffix	Modified	state, agent_name	str
code_util.py	evaluate_code	Modified	state, agent_name	None
debug_util.py	get_code_from_response	Modified	state, agent_name	Response, do_eval
debug_util.py	run_and_debug	New	state, agent_name	Instruction, ...
initialization/agent.py	prepare_task	Modified	state	None
initialization/agent.py	create_workspace	Modified	state, task_id	None
initialization/agent.py	get_task_summary	Modified	state, response	None
initialization/agent.py	get_model_candidates	Modified	state, response, task_id	None
initialization/agent.py	get_model_eval_agent_instruction	Modified	state, agent_name	str
initialization/agent.py	rank_candidate_solutions	Modified	state, task_id	None
initialization/agent.py	run_initialization_pipeline	New	state	None
submission/agent.py	get_submission_and_debug_agent_instruction	Modified	state, agent_name	str
submission/agent.py	run_submission_pipeline	New	state	None
agent.py	run_pipeline	New	task_name, data_dir, AgentState_dir	AgentState_dir
config.py	DefaultConfig	Modified	—	config

This plan prioritizes a working end-to-end pipeline over feature completeness. Once the MVP runs successfully, individual phases (refinement, ensemble, multiple candidates, web search) can be re-enabled incrementally as budget allows.